



Islington college
(इसलिंग्टन कलेज)

Risk Assessment Report

Intelligence Recon System

Name: Prabesh Sundar Taksari

LMU ID: 23047464

1. Introduction

This document analyses the potential risks associated with the development, testing, and operation of the Intelligence Recon System (IRS), an automated web application designed to perform security reconnaissance using artificial intelligence-driven tool orchestration. The goal is to identify security and operational threats, assess their potential impact, and implement appropriate mitigation strategies to ensure the stability, security, and integrity of the IRS platform throughout its development lifecycle.

2. Scope of Risk Assessment

2.1. Target System: Web Application – Intelligence Recon System (IRS)

2.2. Testing Methodology:

1. **Vulnerability Assessment:** Automated scanning of the IRS platform using the integrated tool chain to identify security weaknesses in the application, tool execution pipeline, and output parsing mechanisms.
2. **Functional Validation:** Manual verification of core IRS features including AI-based tool selection logic, scheduled scan triggering, report generation (HTML/PDF), email alert delivery, and dashboard real-time status updates.
3. **Static Code Review:** Inspection of the IRS source code to detect insecure coding practices such as unsafe subprocess invocations, missing input validation, and improper credential handling.
4. **Configuration Review:** Examination of .env files, API key management, database connection settings, and CORS policies to verify that no sensitive information is exposed or misconfigured.
5. **Performance Testing:** Simulation of multiple concurrent scan sessions to evaluate system behavior under load, including database write throughput and dashboard responsiveness.

2.3. Tools Used

Category	Tools
Subdomain Enumeration	Subfinder, Amass, Assetfinder, Sublist3r
DNS Resolution	DNSx, ShuffleDNS
Port Scanning	Naabu
HTTP Probing	Httpx
Web Crawling	Katana, GoSpider, GAU
Fuzzing	Ffuf, Wfuzz, CeWL
Vulnerability Scanning	Nuclei

Table 1 Tool Used

2.4. Testing Environment: Staging/Development (with test targets, full backups and safe configuration) Production is only for controlled validation with permission.

3. Risk Identification

Risk ID	Risk Description	Impact	Likelihood	Mitigation Strategy
R1	AI Decision Node picks wrong tools or skips must-run scans like Nuclei on web vulns, leading to missed threats and bad reports. This hurts scan quality and trust in IRS.	High	Medium	Add manual override button for users to pick 5 default tools (Naabu, Httpx, Nuclei, Subfinder, DNSx). Lock AI prompts to safe version. Log every AI choice with input/output for easy audit and fix later. Test AI on 10+ sample targets.
R2	Gemini API goes down, times out, or hits rate limits mid-scan, stopping AI triage and leaving half-done scans with no results.	High	Medium	Set 30-sec timeouts on API calls with auto-retry up to 3 times using backoff. Switch to fallback mode with fixed tool set if API fails. Show alert on dashboard and log fails for check.

R3	One of 15 tools crashes from missing deps, wrong path, or bad env vars, breaking chain and losing key data like ports or subs.	High	Medium	Run pre-check to see if each tool in PATH works. Wrap every tool runs in try-catch to log error but keep scan going. Save raw output always. Plan version lock for all tools in config.
R4	Raw tool outputs change format or have bad encoding, so parsers fail and structured findings miss from DB, making reports empty.	Medium	Medium	Force UTF-8 to decode with error ignore on all outputs. Save raw files to disk no matter what. Make parsers handle empty/malformed cases. Add unit tests for each parser on real samples.
R5	User inputs like bad domains command injections via subprocess, letting attacks run on server during scans.	High	Low	Strip schemes/whitespace from inputs; check DNS resolve first. Always use subprocess. Exec with arg list, never shell=True. Add regex block for shell chars like; .
R6	API keys, DB pass, SMTP creds leak via logs, error msgs, or Git commit by mistake.	High	Low	Load all from .env gitignore'd. Strip secrets from all logs/responses. Add CORS limits on API. Scan code for hard-coded secrets before committing.
R7	Tool findings are not normalized right, so dups, wrong severity, bad OWASP tags show in charts and reports.	Medium	Medium	Make central schemas for all findings (type, sev, loc, desc). Dedupe by hash before DB insert. Map tool tags to OWASP Top 10 standards. Test on mixed scans.

R8	DB locks up too many writes during big scans, slowing UI or losing data.	Medium	Medium	Use async SQL Alchemy for non-block writes. Set connection pool to 20. Add queue for batch inserts. Backup DB before heavy tests.
R9	Frontend dashboard lags or wrong on live status, tool progress, charts during active scans.	Medium	Medium	Use Server-Sent Events or WebSockets for push updates every 5 sec. Fallback to poll if disconnect. Test with 5 fake concurrent scans.
R10	Scheduled scans (weekly/monthly) fail trigger, run wrong time, or stack up duplicates.	Medium	Low	Pick APScheduler with cron jobs. Log every schedule event. Let users see/edit/cancel from UI. Test all modes on staging.
R11	HTML/PDF reports miss data, bad layout, or fail gen from template bugs or big results.	Medium	Medium	Build one template, test on empty/full scans. Use WeasyPrint for PDF. Auto-check report size/content before download link.
R12	Scans on wrong/unauth targets break laws or ethics, risk for dev/school.	High	Low	Show big warning popup with legal terms. Force user checkbox agrees before starting. Limit docs/tests to lab targets only. Log all targets scanned.

Table 2 Risk Identification

4. Risk Evaluation & Prioritization

Risks are evaluated based on their potential impact and likelihood, prioritizing those that could cause significant security breaches, system failure, or legal violations.

High-priority risks (R1, R2, R3, R5, R6, R12) require immediate mitigation before any testing activity is conducted, as they carry the potential for critical system failure, data breach, command injection, or legal liability.

Medium-priority risks (R4, R7, R8, R9, R10, R11) require proactive handling across subsequent development sprints. While they do not pose an immediate critical threat, they have the potential to degrade report quality, system performance, and user experience.

All identified risks are reviewed and tracked as standing agenda items in weekly Scrum meetings to ensure timely escalation and resolution.

5. Mitigation Strategies

5.1. Pre-Test Measures

- Obtain necessary approvals from the project supervisor and IT administrator.
- Schedule testing during off-peak hours to avoid disruption.
- Take full backups of the application and database before testing.
- Use test accounts and synthetic data only no real user data.
- Notify the security team in advance so that authorized testing traffic can be distinguished from hostile activity.

5.2. During Testing

- Monitor server performance to prevent overloading.
- Ensure logs are captured for review and rollback if necessary.
- Restrict testing scope to pre-approved targets only.
- Stop testing immediately if server load exceeds 80% and document the reason.

5.3. Post-Test Measures

- Review and validate all findings before making any code changes.
- Apply security patches and code-level remediations promptly based on confirmed test results.
- Conduct a follow-up verification scan to confirm no residual issues remain.
- Share a final summary report with all relevant stakeholders.

6. Conclusion

This risk assessment ensures that the development and testing of the Intelligence Recon System is conducted securely without disrupting academic or operational activities. By systematically identifying and mitigating twelve key risks spanning AI reliability, tool chain integrity, data handling, system performance, and legal compliance, the assessment provides a structured governance framework for the project. Implementation of the defined mitigation strategies will safeguard the confidentiality, integrity, and availability of the IRS platform throughout its development lifecycle.